

TOPIC NAME:AI-Powered Resume Ranker

Project Report

Submitted By:

Rachit S Mane

B.Tech – Computer Science Engineering

PES University

Table of Contents

1. Introduction
 2. Problem Statement
 3. Objectives
 4. Technologies Used
 5. System Architecture
 6. Project Workflow
 7. Module Description
 8. Frontend Design
 9. Backend Design
 10. NLP and Machine Learning Approach
 11. Features Implemented
 12. Sample Outputs
 13. Challenges Faced
 14. Future Enhancements
 15. Conclusion
 16. References
-

1. Introduction

Recruitment is one of the most important processes in every organization. Companies receive hundreds or even thousands of resumes for a single job role, making manual screening slow, inefficient, and error-prone.

The AI-Powered Resume Ranker is a web-based application that automates the resume screening process using Natural Language Processing (NLP) and Machine Learning techniques.

The system allows recruiters to:

- Upload multiple resumes in PDF format
- Paste a job description
- Automatically analyze and rank candidates
- View match percentages
- Download HR reports

The project combines Flask backend services with a React frontend interface to create a modern AI-driven recruitment platform.

2. Problem Statement

Traditional resume screening methods require recruiters to manually review resumes, compare skills with job requirements, and shortlist candidates.

This process has several disadvantages:

- Time-consuming
- High chances of human error
- Difficult to manage large numbers of resumes
- Inconsistent evaluation criteria
- Delayed hiring decisions

The objective of this project is to build an intelligent system capable of automatically ranking resumes based on their relevance to a job description.

3. Objectives

The major objectives of the project are:

- Extract text from PDF resumes

- Clean and preprocess resume content using NLP
 - Convert text into numerical vectors using TF-IDF vectorization
 - Calculate similarity scores between resumes and job descriptions
 - Rank candidates automatically
 - Build an interactive modern frontend UI
 - Generate downloadable HR reports
 - Improve recruitment efficiency using AI
-

4. Technologies Used

Frontend

- React.js
- TypeScript
- Tailwind CSS
- GSAP Animations
- Vite

Backend

- Python
- Flask
- Flask-CORS

NLP and Machine Learning

- SpaCy
- Scikit-learn
- TF-IDF Vectorization
- Cosine Similarity

PDF Processing

- pdfplumber

Version Control

- Git

- [GitHub](#)
-

5. System Architecture

The system follows a client-server architecture.

Frontend Responsibilities

- Accept job description input
- Upload resume PDFs
- Display ranked candidates
- Show match scores and keywords
- Download HR reports

Backend Responsibilities

- Extract text from PDFs
- Clean and preprocess text
- Generate vectors
- Calculate similarity scores
- Return ranked results as JSON

Workflow

1. User uploads resumes and enters job description
 2. Frontend sends data to Flask backend
 3. Backend extracts text from PDFs
 4. NLP preprocessing is performed
 5. TF-IDF vectors are generated
 6. Cosine similarity scores are calculated
 7. Results are ranked and returned
 8. Frontend displays ranked candidates
-

6. Project Workflow

Step 1: Resume Upload

The recruiter uploads multiple PDF resumes.

Step 2: Job Description Input

The recruiter enters the job requirements.

Step 3: PDF Text Extraction

The backend extracts text from each uploaded resume using pdfplumber.

Step 4: NLP Preprocessing

The text is cleaned by:

- Removing stop words
- Converting text to lowercase
- Removing punctuation
- Applying lemmatization

Step 5: Vectorization

TF-IDF converts text into numerical vectors.

Step 6: Similarity Calculation

Cosine similarity compares resumes with the job description.

Step 7: Candidate Ranking

Candidates are sorted according to match percentage.

Step 8: Result Display

The frontend displays ranked resumes with match scores and keywords.

7. Module Description

7.1 extractor.py

Responsible for extracting text from uploaded PDF resumes.

Functions:

- Reads PDF files
- Extracts page-wise text
- Returns combined text

7.2 preprocess.py

Responsible for NLP preprocessing.

Functions:

- Text cleaning
- Stop-word removal
- Lemmatization
- Keyword extraction

7.3 vectorizer.py

Responsible for converting textual data into TF-IDF vectors.

7.4 ranker.py

Responsible for calculating cosine similarity scores and ranking resumes.

7.5 app.py

Main Flask backend server.

Responsibilities:

- Handle API requests
- Receive uploaded files
- Process resumes
- Return ranked results

7.6 LiveDemo.tsx

Main frontend ranking dashboard.

Features:

- Resume upload interface
- Ranking dashboard
- Match score visualization
- Download report functionality

8. Frontend Design

The frontend is built using React and TypeScript with a modern AI-inspired UI.

UI Features

- Responsive design
- Animated transitions
- Smooth scrolling using Lenis
- Interactive ranking cards
- Modern dark theme
- Download report button
- Pricing section

Sections Included

- Hero Section
 - Features Section
 - How It Works Section
 - Live Demo Section
 - Testimonials
 - Pricing Section
 - Footer
-

9. Backend Design

The backend is developed using Flask.

API Endpoint

/rank

Accepts:

- Job description
- Multiple resumes

Returns:

- Ranked candidate list
- Match scores
- Extracted keywords

Backend Pipeline

1. Receive files
 2. Extract text
 3. Clean text
 4. Generate vectors
 5. Compute similarity
 6. Return JSON response
-

10. NLP and Machine Learning Approach

Natural Language Processing

SpaCy is used for:

- Tokenization
- Lemmatization
- Stop-word removal
- Text normalization

TF-IDF Vectorization

TF-IDF measures the importance of words in resumes relative to the job description.

Advantages:

- Efficient
- Lightweight
- Accurate for keyword relevance

Cosine Similarity

Cosine similarity measures the angle between vectors.

A higher similarity score indicates a stronger match between a resume and the job description.

11. Features Implemented

Completed Features

- PDF Resume Upload

- NLP Preprocessing
 - TF-IDF Vectorization
 - Resume Ranking
 - Match Percentage Calculation
 - Keyword Matching
 - Interactive Frontend UI
 - Animated Dashboard
 - Download HR Report
 - Responsive Design
 - Modern Pricing Section
 - Candidate Match Visualization
-

12. Sample Outputs

Homepage

The homepage includes:

- AI branding
- Hero section
- Call-to-action buttons
- Feature highlights

Resume Ranking Dashboard

The ranking dashboard displays:

- Candidate names
- Match scores
- Matched keywords
- Best-fit badge
- Ranking cards

Downloadable HR Report

Recruiters can download candidate ranking reports for future use.

13. Challenges Faced

During development, several challenges were encountered:

PDF Extraction Issues

Some PDF resumes had inconsistent formatting.

NLP Accuracy

Cleaning resumes while preserving important technical keywords required careful preprocessing.

Frontend-Backend Integration

Handling file uploads and API communication required debugging and optimization.

UI Responsiveness

Making the interface responsive across different devices required additional styling adjustments.

14. Future Enhancements

The project can be further improved by adding:

- Resume summarization using Generative AI
- Interview question generation
- Candidate recommendation system
- Cloud deployment
- Authentication system
- Database integration
- Email notifications
- Real-time analytics dashboard
- Multi-language support
- AI chatbot assistant

15. Conclusion

The AI-Powered Resume Ranker successfully automates the resume screening process using NLP and Machine Learning.

The project demonstrates how artificial intelligence can improve recruitment efficiency by reducing manual effort and improving candidate shortlisting.

The system provides a complete end-to-end solution consisting of:

- Intelligent resume ranking
- Modern frontend dashboard
- Flask backend API
- NLP preprocessing pipeline
- Downloadable HR reports

This project showcases practical implementation of AI concepts in real-world recruitment systems.

16. References

1. SpaCy Documentation – <https://spacy.io/>
 2. Scikit-learn Documentation – <https://scikit-learn.org/>
 3. Flask Documentation – <https://flask.palletsprojects.com/>
 4. React Documentation – <https://react.dev/>
 5. Tailwind CSS Documentation – <https://tailwindcss.com/>
 6. pdfplumber Documentation – <https://github.com/jsvine/pdfplumber>
 7. GSAP Documentation – <https://greensock.com/gsap/>
 8. Vite Documentation – <https://vitejs.dev/>
-

Appendix

Folder Structure

```
AI-Resume-Ranker/  
|  
├── BACKEND/  
|   ├── app.py  
|   └── extractor.py
```

```
| ├── preprocess.py
| ├── ranker.py
| ├── vectorizer.py
| └── requirements.txt
|
| ├── FRONTEND/
| | ├── src/
| | | ├── sections/
| | | ├── components/
| | | └── App.tsx
| | └── package.json
|
| └── README.md
```